

USB and Linux Driver

Outline

- USB Introduction
- USB Architecture
- USB Framming
- USB Drivers
- URB (USB request Block)

USB : Universal Serial Bus

- Created to replace a wide range of slow and diff bus
 - Parallel, serial, keyboard connections
 - With a single bus type that could connect to
- 480 MBps speed
- USB subsystem not like a bus but tree build out of several point to point link
- These links are 4wires cable (gnd,power, two signal) that connect a device & hub
- The USB host controller is in charge of asking every USB device if it has any data to send
 - This configuration allow for a very easy plug & play type of system, where device can automatically configured

USB protocol standard

- USB protocol spec define a set of standard that any device of a specific type can follow
- If a device follow that standard then a special driver for that device is not necessary
- These device types are called class and consist of thing like
 - Storage, keyboard, mice, joy stick, network, modem
- Other that don't fit require a vendor specific driver Video devices, USB 2 serial devices
- This features inherent hot plug capability of the design, make USB a handy, low cost mechanism to connect/disconnect several devices a runtime

USB History

- Originally developed in 1995 by a consortium including
 - Compaq, HP, Intel, Lucent, Microsoft, and Philips
- USB 1.1 supports
 - Low-speed devices (1.5 Mbps)
 - Full-speed devices (12 Mbps)
- USB 2.0 supports
 - High-speed devices
 - Up to 480 Mbps (a factor of 40 over USB 1.1)
 - Uses the same connectors
 - Transmission speed is negotiated on device-by-device basis

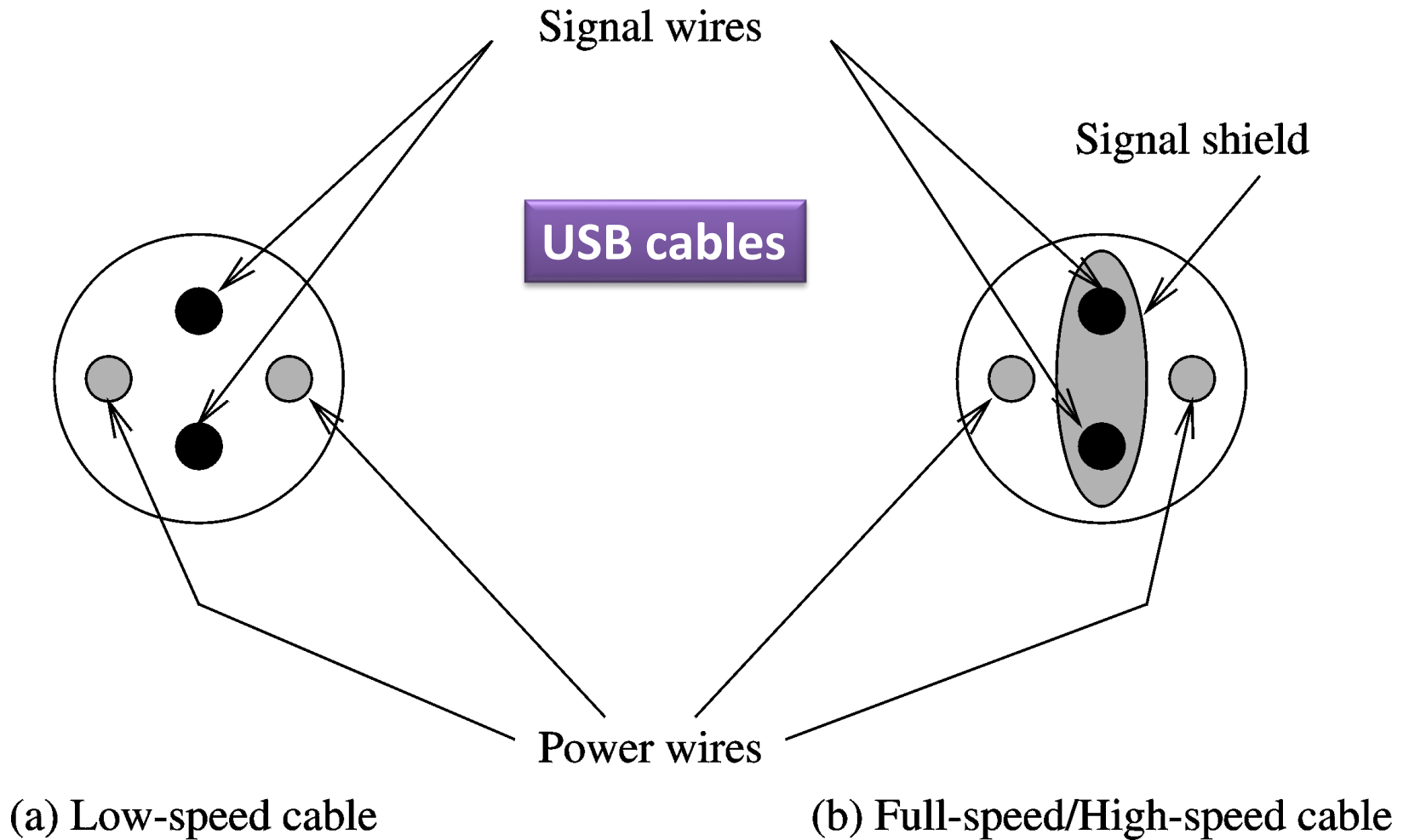
USB Motivation

- Avoid device-specific interfaces
 - Eliminates multitude of interfaces
 - PS/2, serial, parallel, monitor, microphone, keyboard,...
- Avoid non-shareable interfaces
 - Standard interfaces support only one device
- Avoid I/O address space and IRQ problems
 - USB does not require memory or address space
- Avoid installation and configuration problems
 - Don't have to open the box to install and configure jumpers
- Allow hot attachment of devices

Additional advantages of USB

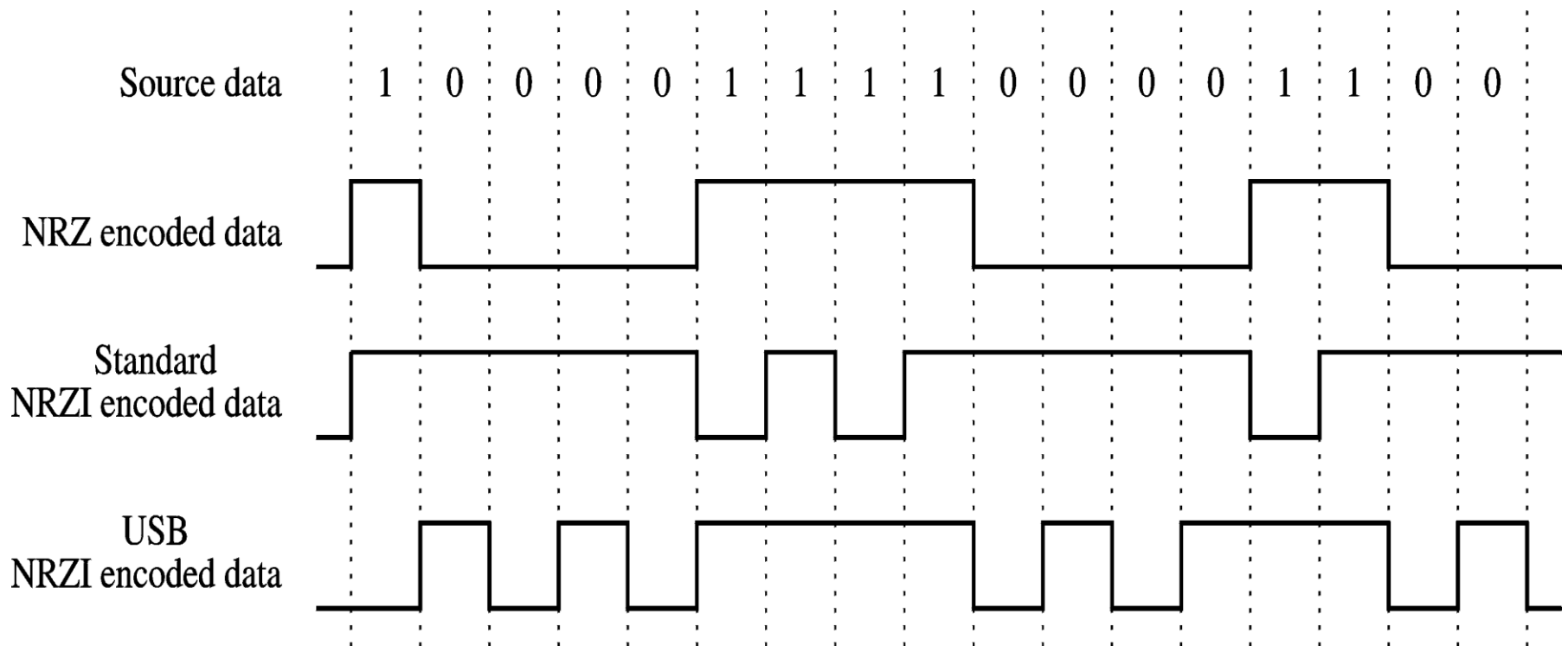
- Power distribution
 - Simple devices can be bus-powered
 - Examples: mouse, keyboards, floppy disk drives, wireless LANs, ...
- Control peripherals
 - Possible because USB allows data to flow in both directions
- Expandable through hubs
- Power conservation
 - Enters suspend state if there is no activity for 3 ms
- Error detection and recovery
 - Uses CRC

USB Cabling



USB Encoding

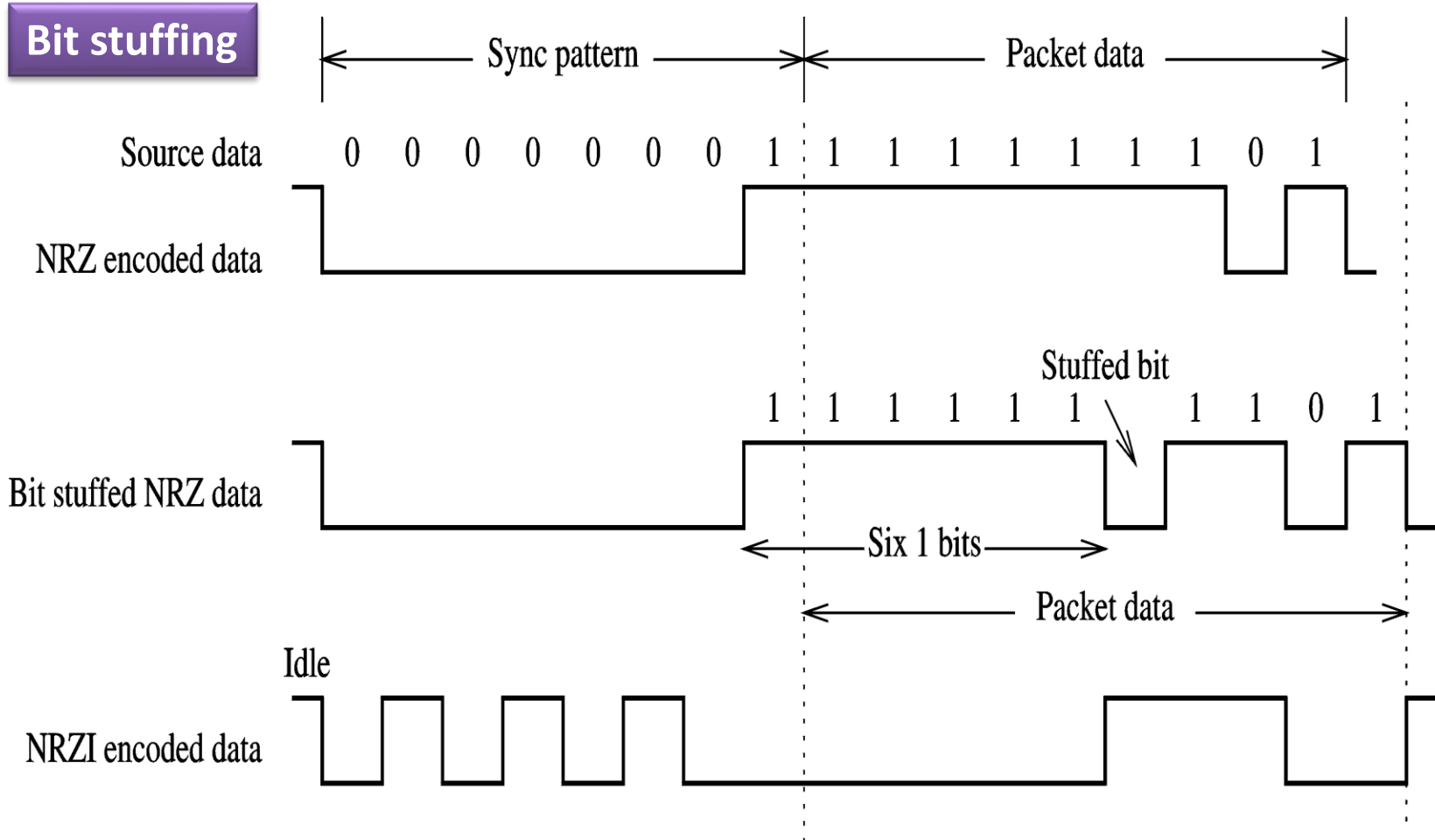
- Uses NRZI encoding
 - Non-Return to Zero-Inverted



USB : NRZI encoding cont'd..

- A signal transition occurs if the next bit is zero
 - It is called ***differential encoding***
- Two desirable properties
 - Signal transitions, not levels, need to be detected
 - Long string of zeros causes signal changes
- Still a problem
 - Long strings of 1s do not causes signal change
- To solve this problem
 - Uses ***bit stuffing***
 - A zero is inserted after every six consecutive 1s

USB Encoding Continued



USB Transfer type

- Transfer types : Four types of transfer
 - Control
 - Interrupt transfer
 - Uses polling
 - Polling interval can range from 1 ms to 255 ms
 - Bulk
 - Isochronous transfer
 - Used in real-time applications that require constant data transfer rate
 - Example: Reading audio from CD-ROM
 - These transfers are scheduled regularly
 - Do not use error detection and recovery

USB : Control transfer

- Used to configure and set up USB devices
- Three phases
 - Setup stage
 - » Conveys type of request made to target device
 - Data stage
 - » Optional stage
 - » Control transfers that require data use this stage
 - Status stage
 - » Checks the status of the operation
- Allocates a guaranteed bandwidth of 10%
- Error detection and recovery are used
 - Recovery is by means of retries

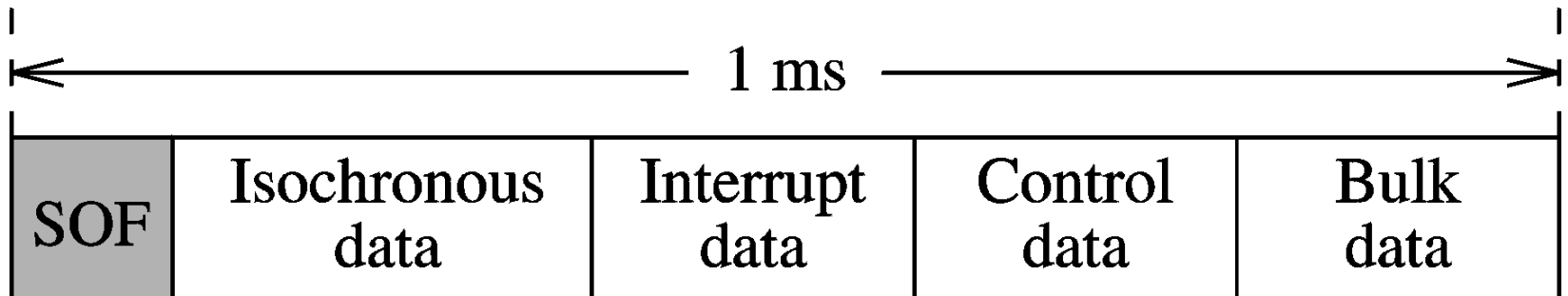
USB : Bulk Transfer

- For devices with no specific data transfer rate requirements
 - Example: sending data to a printer
- Lowest priority bandwidth allocation
- If the other three types of transfers take 100% of the bandwidth
 - Bulk transfers are deferred until load decreases
- Error detection and recovery are used
 - Recovery is by means of retries

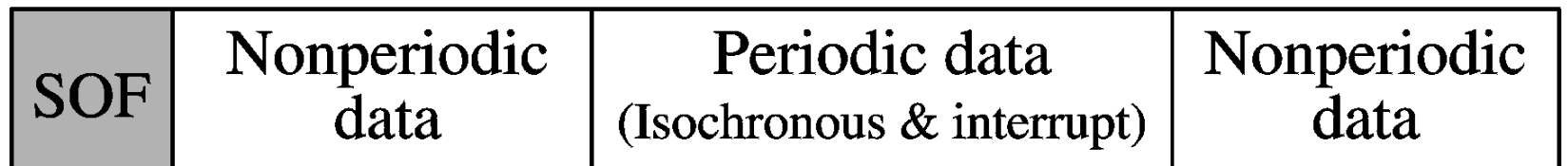
USB : USB architecture

- USB host controller
 - Initiates transactions over USB
- Root hub
 - Provides connection points
- Two types of host controllers
 - Open host controller (OHC)
 - Defined by Intel
 - Universal host controller (UHC)
 - Specified by National Semiconductor, Microsoft, Compaq
 - Difference between the two
 - How they schedule the four types of transfers

USB Scheduling



(a) UHC scheduling



(b) OHC scheduling

USB: UHC Scheduling

- Universal Host Controller
- Schedules periodic transfers first
 - Periodic transfers: isochronous and interrupts
 - Can take up to 90% of bandwidth
- These transfers are followed by control and bulk transfers
 - Control transfers are guaranteed 10% of bandwidth
- Bulk transfers are scheduled only if there is bandwidth available

USB: OHC Scheduling

- Open Host Controller
- Different from UHC scheduling
- Reserves space for non-periodic transfers first
 - Non-periodic transfers: control and bulk
 - 10% bandwidth reserved
- Next periodic transfers are scheduled
 - Guarantees 90% bandwidth
- Left over bandwidth is allocated to non-periodic transfers

USB: Bus Powered Device

– Low-power

- Less than 100 mA
- Can be bus-powered

– High-powered

- Between 100 mA and 500 mA
 - Full-powered ports can power these devices
- Can be designed to have their own power
- Operate in three modes
 - Configured (500 mA)
 - Unconfigured (100 mA)
 - Suspended (about 2.5 mA)

USB Hubs

— Bus-powered

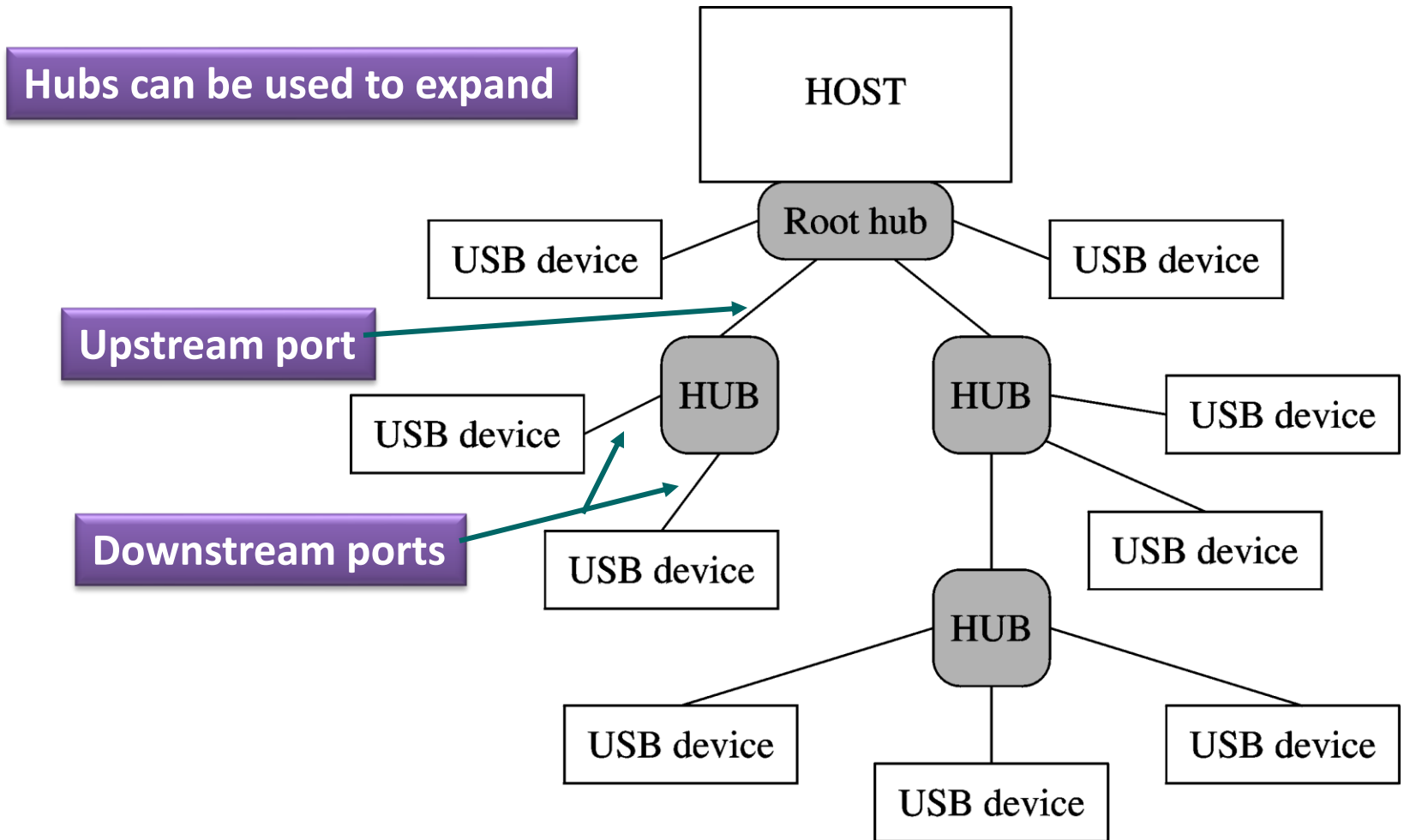
- No extra power supply required
- Must be connected to an upstream port that can supply 500 mA
- Downstream ports can only supply 100 mA
 - Number of ports is limited to four
 - Support only low-powered devices

— Self-powered

- Support 4 high-powered devices
- Support 4 bus-powered USB hubs

— Most 4-port hubs are dual-powered

USB Tree

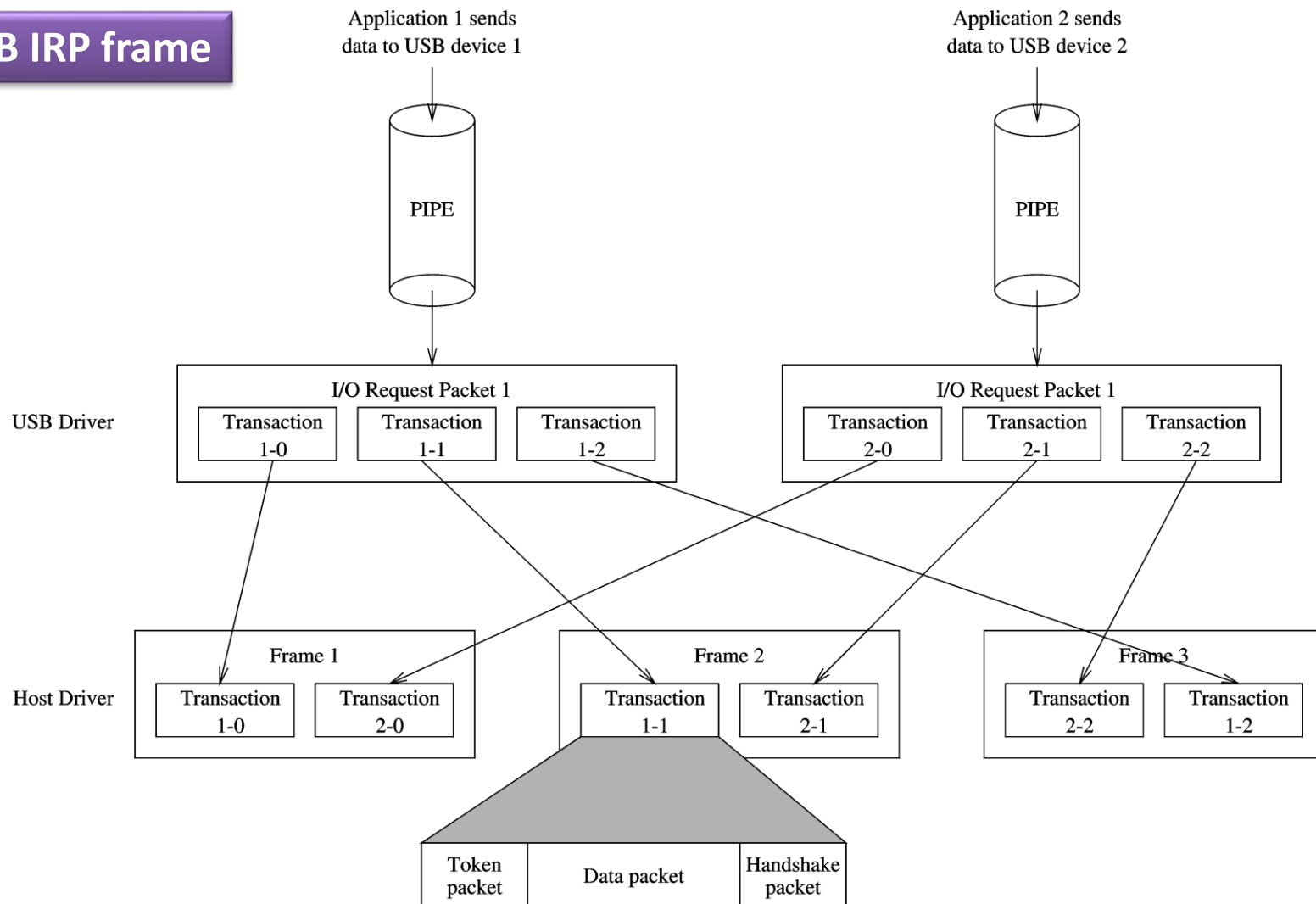


USB transactions

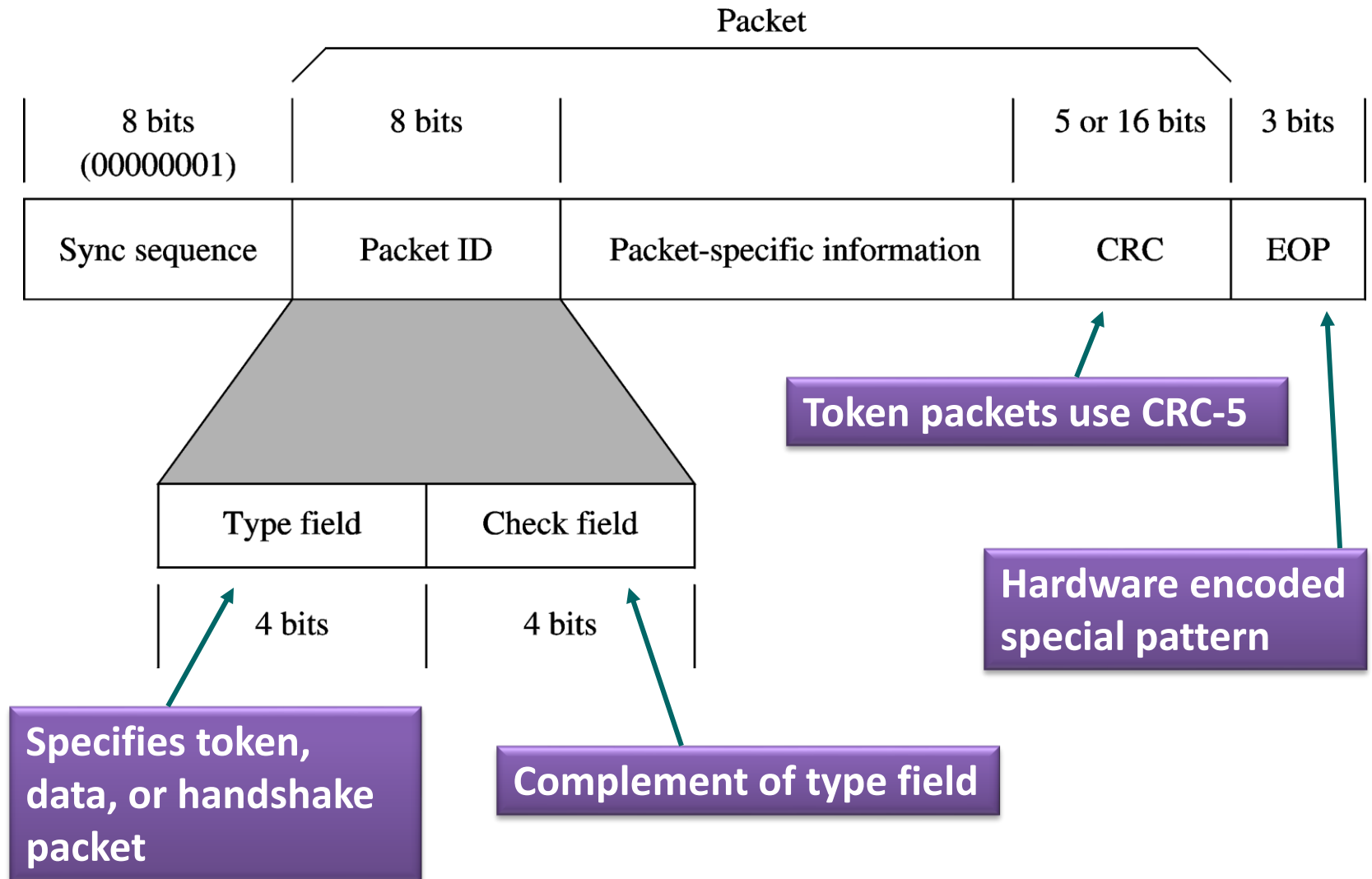
- Transfers are done in one or more transactions
 - Each transaction consists of several packets
- Transactions may have between 1 and 3 phases
 - Token packet phase
 - Specifies transaction type and target device address
 - Data packet phase (optional)
 - Maximum of 1023 bytes are transferred
 - Handshake packet phase
 - Except for isochronous transfers, others use error detection for guaranteed delivery
 - Provides feedback on whether data has been received without error

USB IRP (IO Req Packet)

USB IRP frame

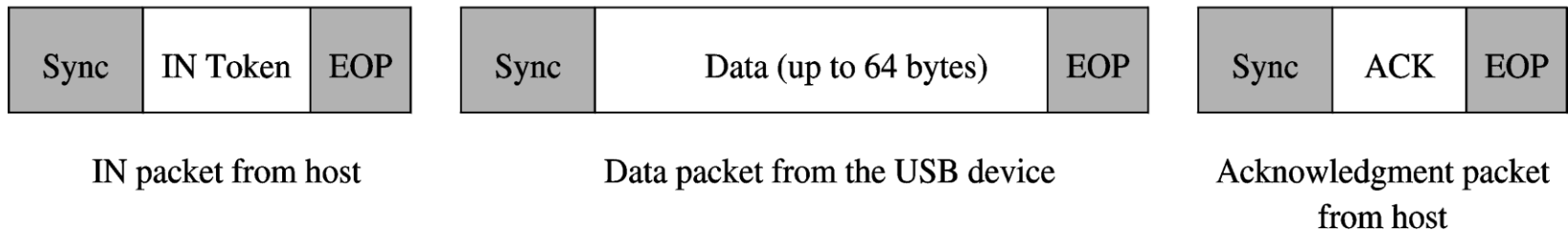


USB Frame



USB Frame

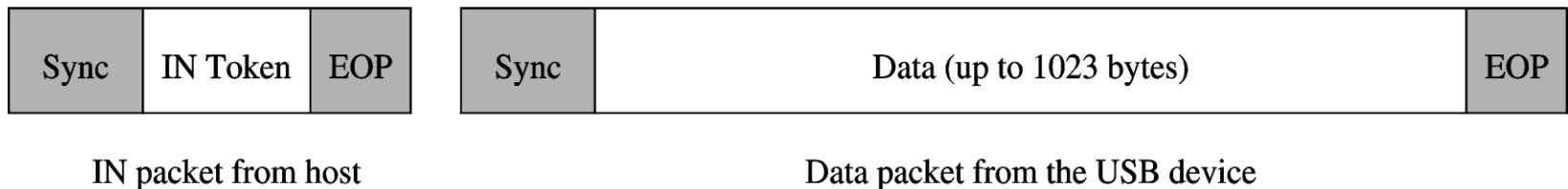
One transaction



USB 1.1 transactions

(a) IN transaction without errors

One transaction



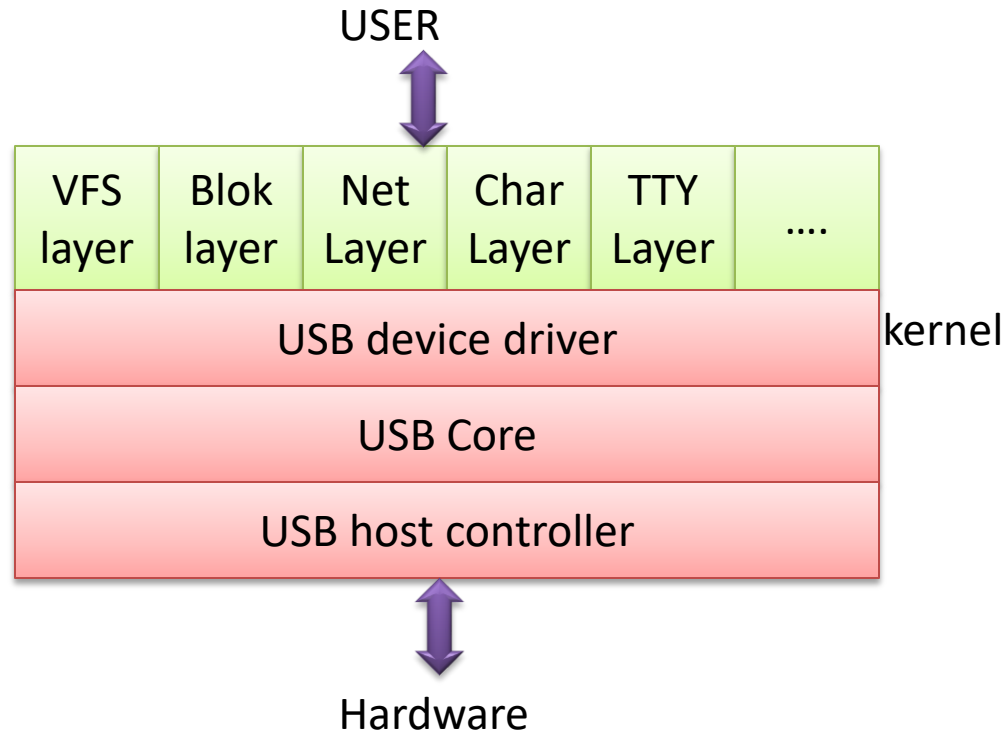
(b) IN transaction during an isochronous transfer

USB 2.0

- USB 2.0
 - USB 1.1 uses 1 ms frames
 - USB 2.0 uses 125 μ s frames
 - 1/8 of USB 1.1
 - Supports 40X data rates
 - Up to 480 Mbps
 - Competitive with
 - SCSI
 - IEEE 1394 (FireWire)
 - Widely available now

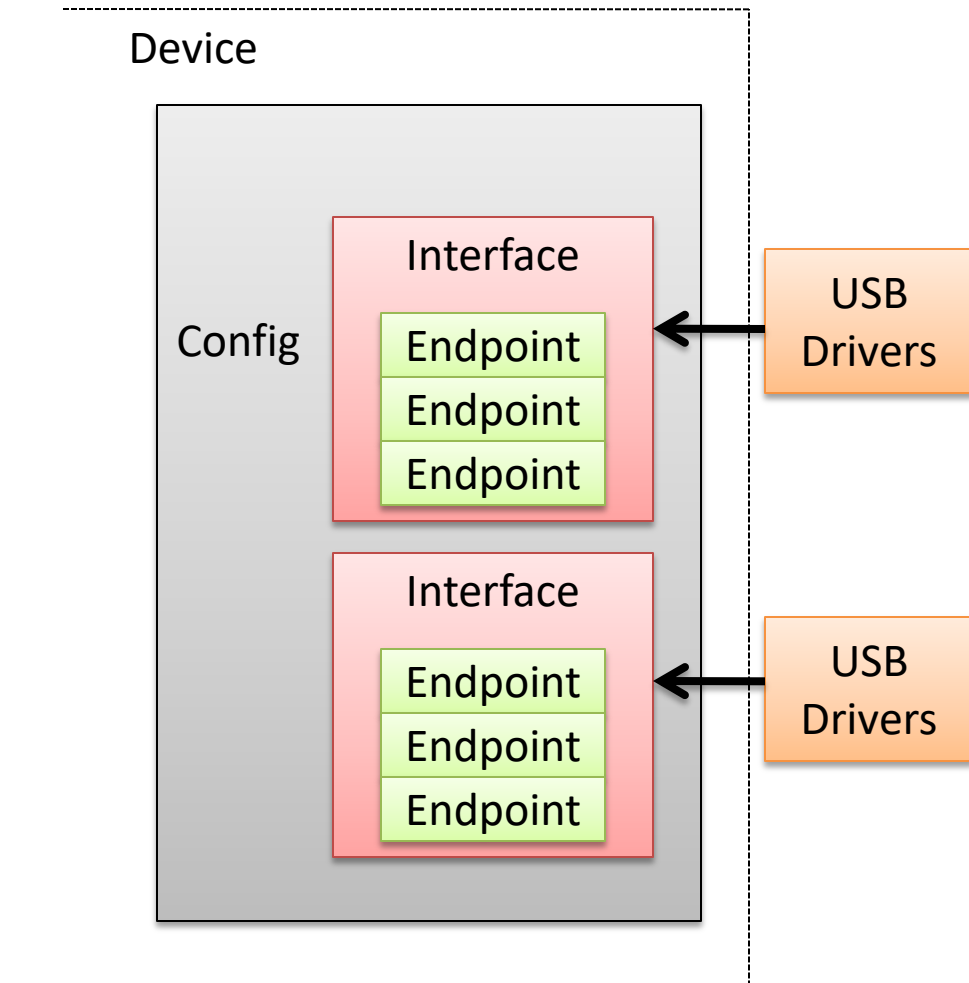
USB driver over view

- USB drivers lives between different kernel subsystem (blk,net,char..) and USB HW controller
- USB core provide an interface to access for USB drivers to use to access and control USB hw
- Connect without worrying the different types of USB hardware controller that are present in the system



USB Device Basic

- USB device is complex thing
- Linux kernel provides a subsystem USB Core to handle most of the complexity
- USB device consist of **configuration, interface, endpoints**
- USB device bind to USB interfaces not to entire USB devices



USB: Endpoints

- USB endpoint can carry data in only one direction either host to dev (OUT endpoint) or host to dev (IN endpoint)
- Endpoints are 4 types: that describe how data is transmitted
 - Control: allow access to diff parts of USB dev. Used for conf dev, retrieve info, send cmd, retrieve status. This is small size end points call endpoint 0
 - Interrupt:small amount of data at fixed rate: keyborad/mice
 - BULK:Large data with no loss : printer, storage, network
 - ISOCHRONOUS: large data, not guaranteed, stream audio/video
- Struct `usb_host_endpoint`: address, bitmask attaributes, Maxpacket size that point can handle , Intr Interval

USB Interfaces

- USB end points are bundled up with interfaces
- One interface handle one type of logical connection (a kbd,mice or a audio)
- Some USB can handle multiple interfaces USB speaker with kbd for buttons and USB audio stream
- USB interfaces may have alternate setting which are diff choices for param of the interface
- Struct usb_interface: array of altsetting, num_altsetting, cur_altsetting, minor

USB Configuration

- USB interface are themselves bundle up with configurations
- A USB device can have multiple conf and might switch between them inorder to change the state of device
- A single conf is activated at a time
- Summery:
 - Device: one or more Conf
 - Conf : one or more Interface
 - Interface: one or more setting
 - Interface: one or more end points

USB & Sysfs

- `/sys/dev/pci0000:00/000:9:0/usb2/2-1`
- Long device path
- First USB device is a root hub: USB controller usually connected in a PCI device
- Whole USB bus connect to root hub
- Every USB device take number of root hub as first number in its name, followed by character and then the number of port that device connected.
- `Root_hub-hub_port:config.interface`

USB urb (USB Request Block)

- Linux kernel communicated with all USB device with URBS
- Urb is used to send/receive data from a specific USB endpoints on a specific usb device in async manner
- Every end point can handle queue of urbs
- Lifecycle of urb
 - Created by USB dev driver
 - Assigned to specific endpoints of a USB dev
 - Submitted to a USB core by USB driver
 - Submitted to USB host Controller by USB core
 - Processed by the USB Host Controller that makes a USB transfer to the device
 - When urbb is completed, USB host controller notify the dev driver